Aufgabe: Interfaces

Für: Michael **Referenz:** W3Schools – C# Interface **Bearbeitungszeit:** ca. 45–60 Min. **Voraussetzung:** Diese Aufgabe baut auf deiner **Abstraktions-Aufgabe** (`abstract class Fahrzeug` , `Auto` , `Motorrad` , `Fahrrad`) auf. Neu ist **Interface**. **Lernziel:** Am Ende verstehst du, was ein Interface ist, wie es sich von einer abstrakten Klasse unterscheidet, und kannst es selbst einsetzen, um unabhängige Klassen mit einer gemeinsamen Fähigkeit auszustatten.

---

1. Was du schon kannst (kurzer Rückblick)

- **Abstrakte Klasse:** `abstract class Fahrzeug` – man kann kein Objekt direkt davon erzeugen, nur davon erben
- **Abstrakte Methode:** `public abstract void Hupen();` – kein Body, jede Kindklasse **muss** sie überschreiben
- **Vererbung + Polymorphie:** `Auto : Fahrzeug` , `List<Fahrzeug>` mit `foreach` , jedes Objekt verhält sich passend zu seinem eigenen Typ

2. Eine Lücke, auf die du stossen wirst

Stell dir vor, du willst ein `ElektroAuto` bauen, das **aufgeladen** werden kann. Und irgendwann vielleicht auch eine ganz andere Klasse, z.B. `Smartphone` – die ebenfalls aufgeladen werden kann, aber **kein** Fahrzeug ist.

Leitfrage, bevor du weiterliest: Könnte `Smartphone` von `Fahrzeug` erben, nur um `Aufladen()` zu bekommen? Überlege selbst: Ergibt das inhaltlich Sinn? Und erinnere dich an eine Einschränkung von Vererbung, die du bereits kennst – könnte sie hier zusätzlich zum Problem werden?

Für genau diese Art von Problem gibt es Interfaces.

3. Neu: Interface

Lies zuerst das W3Schools-Kapitel: **C# Interface**.

Die Kernidee laut W3Schools: *“An interface is a completely ‘abstract class’, which can only contain abstract methods and properties (with empty bodies).”* Ein Interface ist damit die zweite Möglichkeit (neben abstrakten Klassen), Abstraktion zu erreichen.

Wichtige Fakten aus dem Kapitel: - Konvention: Interface-Namen beginnen mit `I` (z.B. `IAnimal`), damit man sie sofort erkennt - Mitglieder eines Interfaces sind standardmässig `abstract` und `public` - Ein Interface kann Methoden und Properties enthalten, **aber keine Felder** - Ein Interface kann **keinen Konstruktor** haben (man kann ja kein Objekt davon erzeugen) - Implementiert wird mit `:` – genau wie bei Vererbung - Beim Implementieren brauchst du **kein** `override` (anders als bei `abstract class` !) - Eine Klasse kann **mehrere** Interfaces implementieren – das ist der Ersatz für die “multiple inheritance”, die C# bei Klassen nicht erlaubt

Das W3Schools-Beispiel

```
interface IAnimal
{
  void animalSound();  // Interface-Methode, kein Body
}

class Pig : IAnimal
```

```
{
  public void animalSound()   // kein "override" nötig!
  {
    Console.WriteLine("The pig says: wee wee");
  }
}
```

Vergleichstabelle – trag sie für dich selbst fertig aus, während du das Kapitel liest:

	`abstract class`	`interface`
Objekt direkt erzeugbar?	Nein	Nein
Felder erlaubt?	Ja	?
Normale (fertige) Methoden erlaubt?	Ja	?
Konstruktor erlaubt?	Ja	?
Eine Klasse kann davon wie viele haben?	nur 1 (Basisklasse)	?
`override` beim Implementieren nötig?	Ja	?

---

4. Deine Aufgabe: `IAufladbar` für Fahrzeuge und mehr

Schritt 1 – Interface definieren

Definiere ein Interface `IAufladbar` mit einer Methode `Aufladen()` (kein Rückgabewert).

Schritt 2 – Erste Klasse: kombiniere Vererbung UND Interface

Baue eine neue Klasse `ElektroAuto`, die **gleichzeitig**: - von deiner bestehenden `abstract class Fahrzeug` erbt (inklusive `override` von `Hupen()`, wie du es schon kennst) - das Interface `IAufladbar` implementiert (`Aufladen()`, ohne `override`)

```
class ElektroAuto : Fahrzeug, IAufladbar
{
    // dein Code
}
```

Leitfrage: Warum steht hier nur ein `:`, aber danach zwei Namen mit Komma getrennt? Was unterscheidet `Fahrzeug` von `IAufladbar` in dieser Zeile?

Schritt 3 – Eine völlig unabhängige zweite Klasse

Baue eine Klasse `Smartphone`, die **nicht** von `Fahrzeug` erbt (sie hat mit deiner Fahrzeug-Hierarchie nichts zu tun), aber **ebenfalls** `IAufladbar` implementiert.

Leitfrage: Was haben `ElektroAuto` und `Smartphone` jetzt gemeinsam, obwohl sie inhaltlich nichts miteinander zu tun haben?

Schritt 4 – Testen

```
IAufladbar meinAuto = new ElektroAuto(/* ... */);
IAufladbar meinHandy = new Smartphone(/* ... */);

meinAuto.Aufladen();
meinHandy.Aufladen();
```

Vorhersage zuerst: Funktioniert das? Was gibt es aus? Notiere deine Vermutung, bevor du testest.

Schritt 5 – Die eigentliche Stärke: eine gemischte Liste

```
List<IAufladbar> aufladbareGeraete = new List<IAufladbar>();
aufladbareGeraete.Add(new ElektroAuto(/* ... */));
aufladbareGeraete.Add(new Smartphone(/* ... */));

foreach (IAufladbar gerät in aufladbareGeraete)
{
    gerät.Aufladen();
}
```

Leitfrage: Vergleiche das mit deiner `List<Fahrzeug>` aus der letzten Aufgabe. Was ist strukturell ähnlich? Was ist der entscheidende Unterschied – welche Beziehung besteht zwischen `ElektroAuto` und `Smartphone`, die es bei `Auto` / `Motorrad` / `Fahrrad` nicht gab?

Schritt 6 – Fehler bewusst provozieren

Versuch, direkt ein Objekt vom Interface zu erzeugen:

```
IAufladbar irgendwas = new IAufladbar();
```

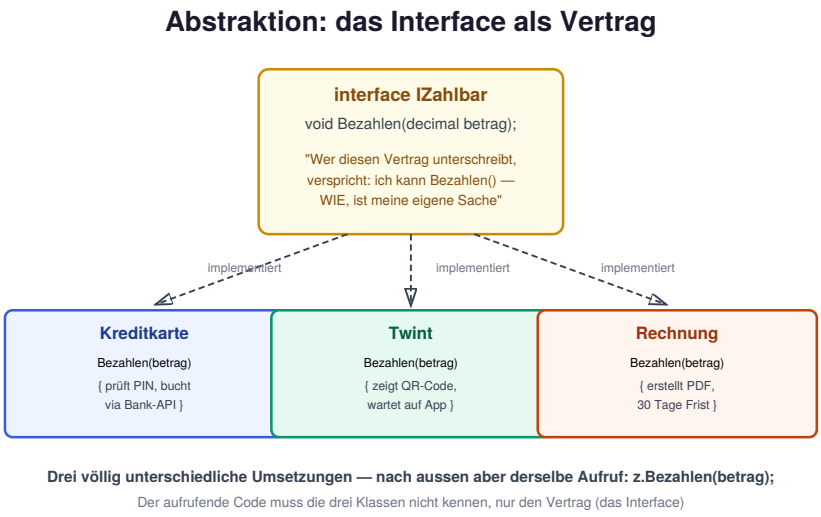
Lies die Fehlermeldung genau. Vergleiche sie gedanklich mit der Fehlermeldung, die du bei `new Fahrzeug(...)` in der Abstraktions-Aufgabe bekommen hast.

---

5. Stolperstein-Check

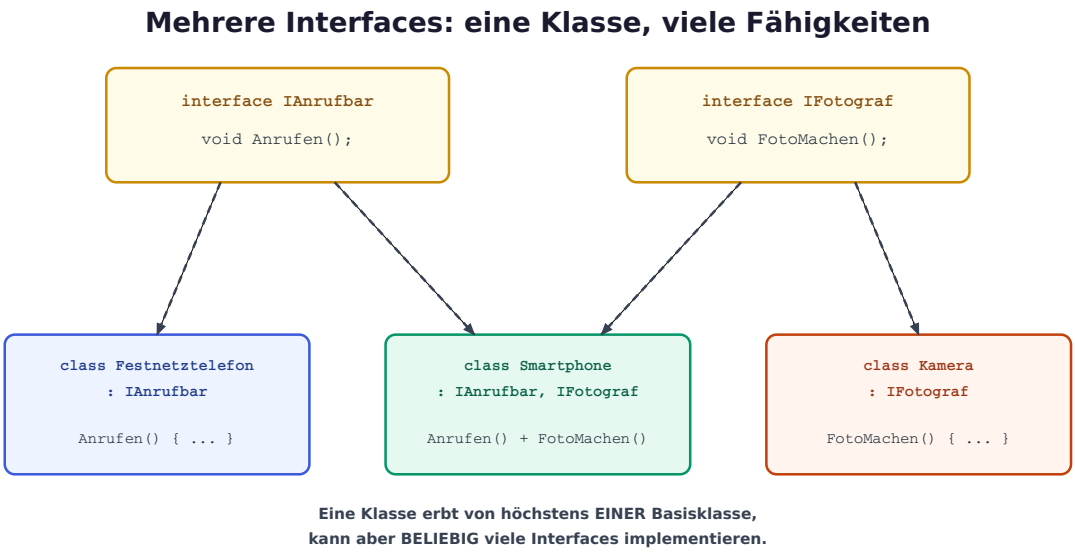
- ☐ Hast du bei `ElektroAuto` sowohl von `Fahrzeug` geerbt **als auch** `IAufladbar` implementiert – erkennst du im Code, welcher der beiden Namen die Basisklasse und welcher das Interface ist?
- ☐ Hast du bei `Aufladen()` **kein** `override` geschrieben? (Prüfe: Bei `Hupen()` aus der Abstraktions-Aufgabe brauchtest du `override`, bei einer Interface-Methode nicht.)
- ☐ Hast du versucht, ein Feld (z.B. `private int _ladezustand;`) direkt im Interface zu deklarieren? Was sagt der Compiler dazu?
- ☐ Hast du geprüft, ob dein Interface einen Konstruktor haben darf?

6. Bilder zur Orientierung



Interface als Vertrag

Leitfragen zum Bild: Was ist bei den drei Umsetzungen unterschiedlich, was gleich? Warum reicht dem aufrufenden Code der Vertrag (das Interface), ohne die einzelnen Klassen zu kennen?



Mehrere Interfaces

Leitfrage zum Bild: Wie viele Basisklassen kann eine Klasse laut diesem Bild maximal haben, wie viele Interfaces?

---

7. Transferaufgabe (eigenständig, ohne Vorlage)

Baue **ohne Hilfe** ein eigenes Interface zu einem Thema deiner Wahl (z.B. `IDruckbar` mit `Drucken()`, implementiert von zwei völlig unabhängigen Klassen wie `Dokument` und `Foto`; oder `ISpeicherbar` mit `Speichern()`). Anforderungen:

- Ein Interface mit mindestens einer Methode
- Mindestens zwei Klassen, die das Interface implementieren und **nicht** in einer Vererbungsbeziehung zueinander stehen
- Eine `List<DeinInterface>` mit `foreach`, die beide Klassen polymorph behandelt
- Mindestens eine Klasse, die **gleichzeitig** von einer Basisklasse erbt **und** dein Interface implementiert (wie `ElektroAuto` in dieser Aufgabe)

8. Abschluss-Fragen (mündlich beantworten)

1. Was ist der Unterschied zwischen einer abstrakten Klasse und einem Interface – nenn mindestens zwei Punkte aus deiner Vergleichstabelle aus Abschnitt 3?
2. Warum brauchst du beim Implementieren eines Interfaces kein `override`?
3. Warum kann `Smartphone` dasselbe Interface implementieren wie `ElektroAuto`, obwohl beide Klassen nichts miteinander zu tun haben?
4. Was passiert, wenn du versuchst, ein Feld direkt in einem Interface zu deklarieren?
5. Wie viele Basisklassen kann eine Klasse haben, wie viele Interfaces?

9. Weiterführend

Der nächste Schritt: **C# Multiple Interfaces** – dort siehst du, wie eine Klasse gleich mehrere Interfaces gleichzeitig implementiert, mit Komma getrennt.